Final Project Viva Preparation Notes

Basic Concepts of OOP

Object-Oriented Programming (OOP) is a type of programming added to php5 that makes building complex, modular and reusable web applications that much easier.

Class : This is a programmer-defined data type, which includes local functions as well as local data. You can think of a class as a template for making many instances of the same kind (or class) of object.

Object An individual instance of the data structure defined by a class. You define a class once and then make many objects that belong to it. Objects are also known as instance.

Classes and objects are separate but related concepts. Every object belongs to a class and every class contains one or more related objects.

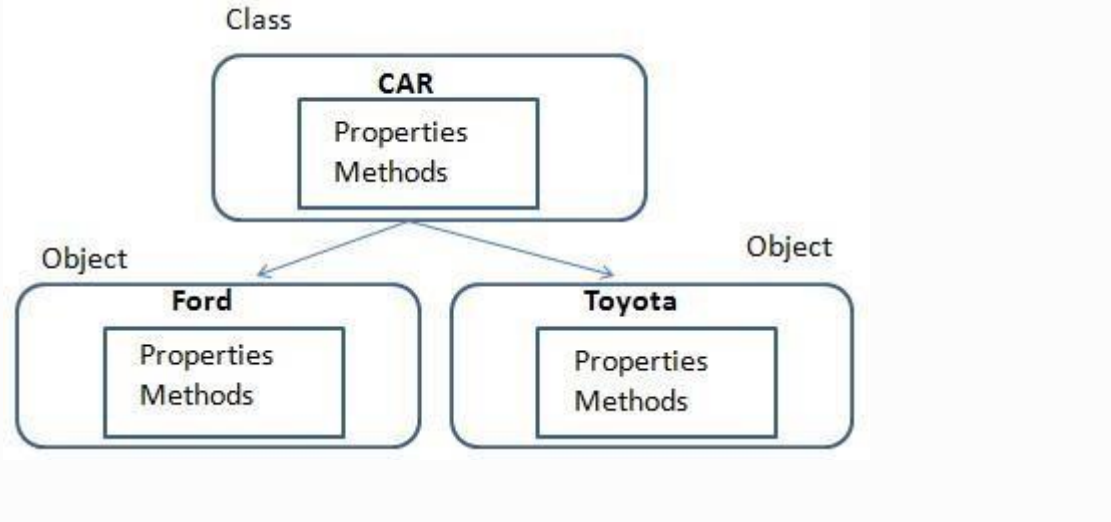
What is the Difference Between a Class and an Object?

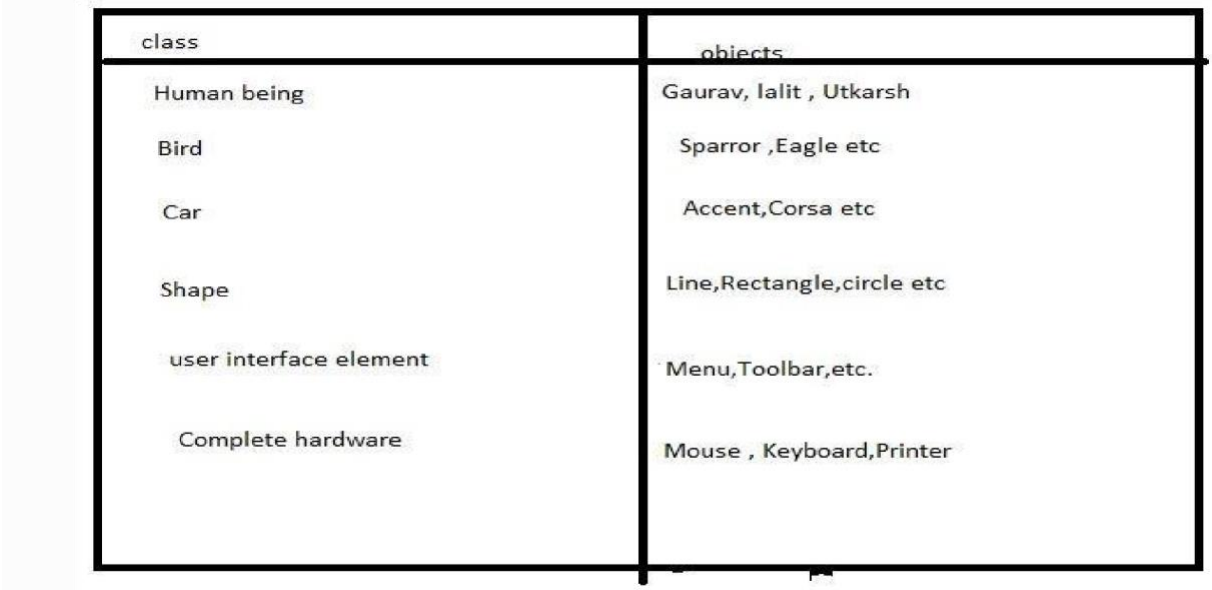
So what exactly are classes and objects and what is the difference between them?

A Class is static. All of the attributes of a class are fixed before, during, and after the execution of a program. The attributes of a class don't change.

The class to which an object belongs is also (usually) static. If a particular object belongs to a certain class at the time that it is created then it almost certainly will still belong to that class right up until the time that it is destroyed.

An Object on the other hand has a limited lifespan. Objects are created and eventually destroyed. Also during that lifetime, the attributes of the object may undergo significant change.





class	objects
Human being	Gaurav, lalit , Utkarsh
Bird	Sparror ,Eagle etc
Car	Accent,Corsa etc
Shape	Line,Rectangle,circle etc
user interface element	Menu,Toolbar,etc.
Complete hardware	Mouse , Keyboard,Printer

Class:

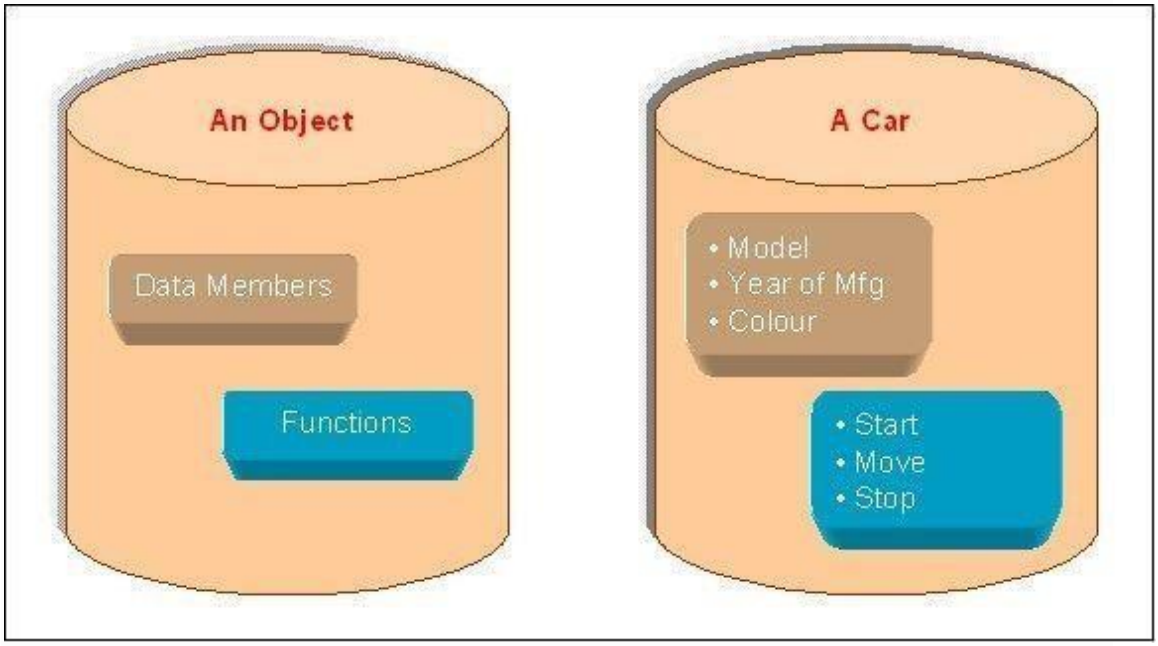
A class is the core of any modern Object Oriented Programming language Class is a blueprint of an object that contains variables for storing data and functions to performing operations on these data.

Object:

Objects are the basic run-time entities in an object oriented system. They may represent a
person, a place or any item that the program has to handle.

"Object is a Software bundle of related variable and methods. "
“Object is an instance of a

class" what are data member
and functions



OOP : Abstraction, Encapsulation, Inheritance, Polymorphism

Abstraction is the "process of representing only essential features". That means Abstraction doesn't show the complexity behind features. Abstraction is used for

"Making things more general, simpler, and abstract".

Real Time Example for Abstraction:

The Best real time example for abstraction is ATM (Automated Teller Machine). We doesnt know how ATM internally works when we are using ATM, we know only select options like withdraw, Balance Inquiry, Mini Statement etc. Here abstraction hides all unnecessary things, it shows only necessary things.

Encapsulation is defined as “Wrapping up data member and method together into a single unit”.

Real time example of Encapsulation is Car Driving, car driver knows how to start car by pressing start button. The driver doesn't know what happened inside when pressing start button. Here the starting process is hidden from driver. So this process can be called as “starting process is encapsulated from driver”

Polymorphism name itself tells “many forms”. That is Poly means “Many” morphism means “forms”. So Polymorphism meaning in Oop “one name. Or Polymorphism can also define as “same operation may behave differently on different classes”.

Real Time Example for Polymorphism Real time example for polymorphism is “Door”, why mean we can use these doors for home, car, lift etc.

Inheritance is defined as “one class (child class) inherits or acquire the property (members) of another class” Real time example for Inheritance is “parent child relationship” Because child receives all the property from parent.

Parent class : A class that is inherited from by another class. This is also called a base class or super class.

Child Class: A class that inherits from another class. This is also called a subclass or derived class

OOP: Association, Aggregation, and Composition

Association is a relationship where all objects have their own lifecycle and there is no owner.

Let's take an example of Teacher and Student. Multiple students can associate with single teacher and single student can associate with multiple teachers, but there is no ownership between the objects and both have their own lifecycle. Both can be created and deleted independently.

Aggregation is a specialized form of Association where all objects have their own lifecycle, but there is ownership and child objects can not belong to another parent object.

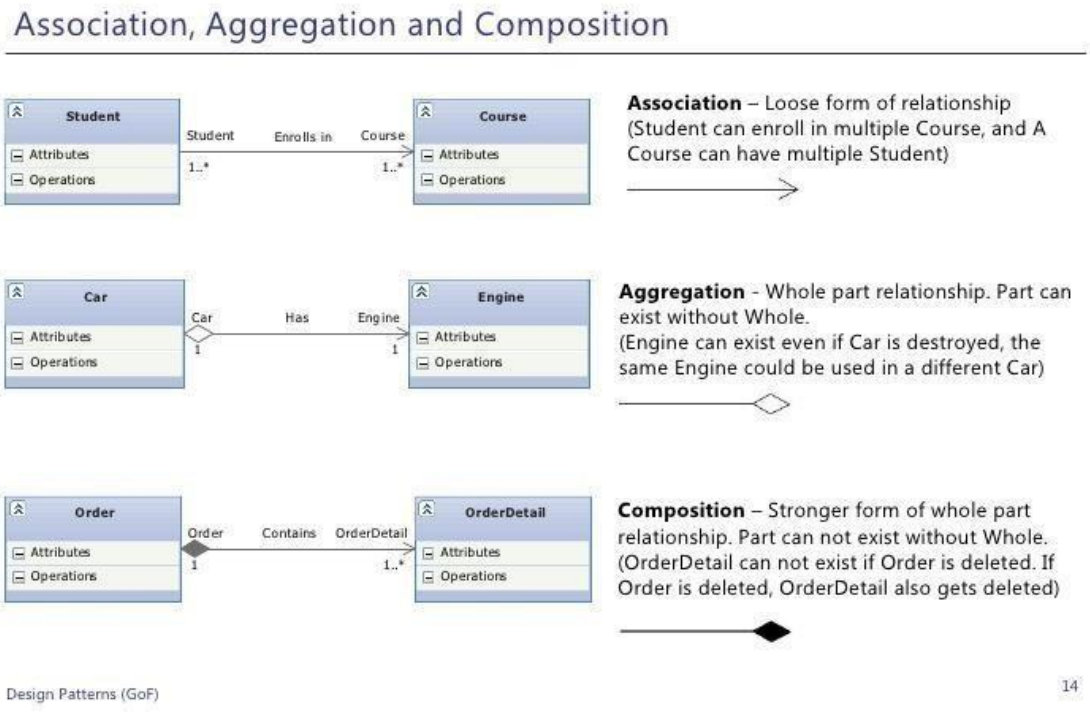
Let's take an example of Department and teacher. A single teacher can not belong to multiple departments, but if we delete the department, the teacher object will not be destroyed. We can think about it as a "has-a" relationship.

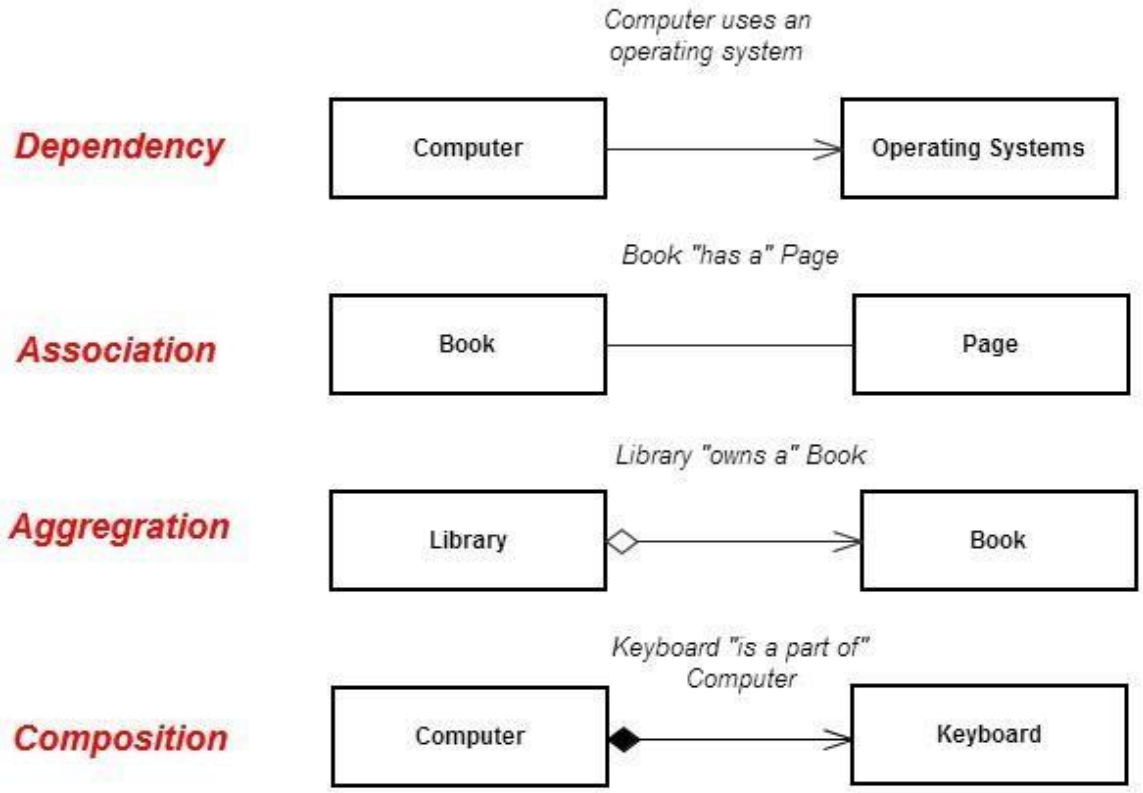
Composition is again specialized form of Aggregation and we can call this as a "death" relationship. It is a strong type of Aggregation.

Child object does not have its lifecycle and if parent object is deleted, all child objects will also be deleted.

Let's take again an example of relationship between House and Rooms. House can contain multiple rooms - there is no independent life of room and any room can not belong to two different houses. If we delete the house - room will automatically be deleted.

Let's take another example relationship between Questions and Options. Single questions can have multiple options and option can not belong to multiple questions. If we delete the questions, options will automatically be deleted.





Member Variable : These are the variables defined inside a class. This data will be invisible to the outside of the class and can be accessed via member functions. These variables are called attribute of the object once an object is created.

Member function : These are the function defined inside a class and are used to access object data.

Overloading : a type of polymorphism in which some or all of operators have different implementations depending on the types of their arguments. Similarly functions can also be overloaded with different implementation.

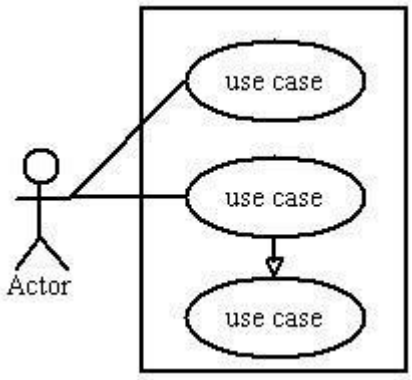
Overriding means to override the functionality of an existing method.

Constructor : refers to a special type of function which will be called automatically whenever there is an object formation from a class.

Destructor : refers to a special type of function which will be called automatically whenever an object is deleted or goes out of scope.

Use Case Diagram

A use case diagram at its simplest is a representation of a user's interaction with the system that shows the relationship between the user and the different use cases in which the user is involved.

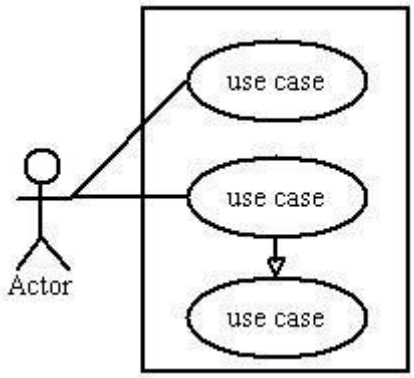


Basic Use Case Diagram Symbols and Notations

System

Draw your system's boundaries using a rectangle that contains use cases. Place actors outside the system's boundaries.

System is used to define the scope of the use case and drawn as a rectangle. This an optional element but useful when your visualizing large systems. For example you can create all the use cases and then use the system object to define the scope covered by your project. Or you can even use it to show the different areas covered in different releases.



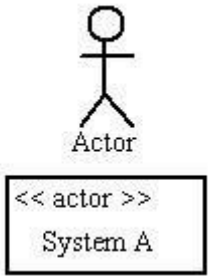
Use Case

Draw use cases using ovals. Label with ovals with verbs that represent the system's functions.



Actors

Actors are the users of a system. When one system is the actor of another system, label the actor system with the actor stereotype.

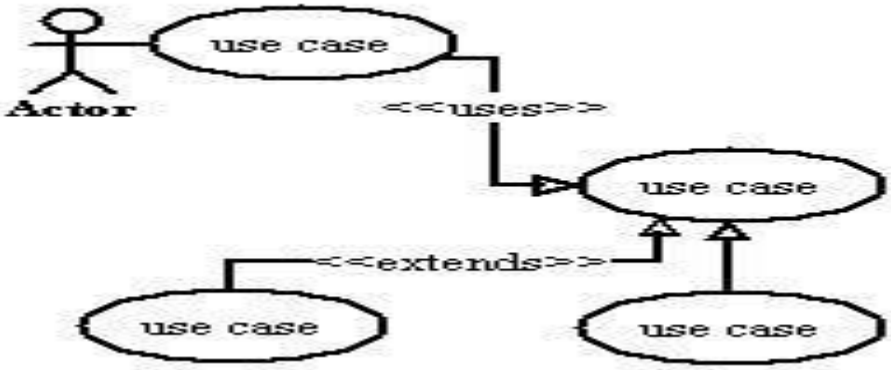


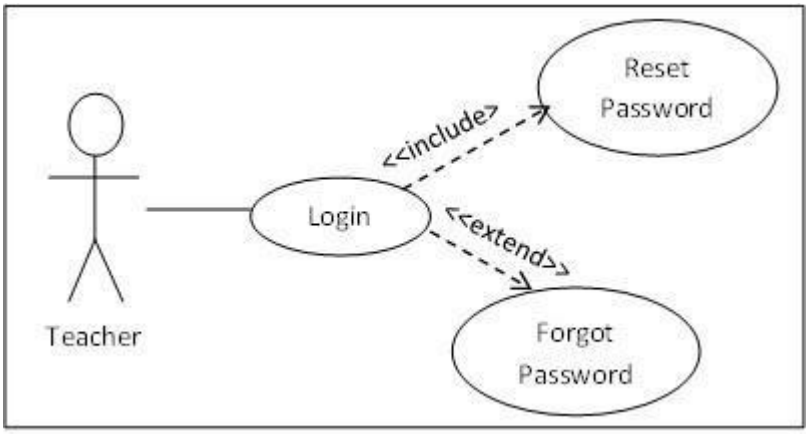
Relationships

Illustrate relationships between an actor and a use case with a simple line. For relationships among use cases, use arrows labeled either "uses" or "extends" or "includes

A "uses" relationship indicates that one use case is needed by another in order to perform a task.
An "extends" relationship indicates alternative options under a certain use case.

An include relationship is a relationship between two use cases It indicates that the use case to which the arrow points is included in the use case on the other side of the arrow. This makes it possible to reuse a use case in another use case.





The following is a sample use case diagram representing the order management system. So if we look into the diagram then we will find three use cases (Order, Special Order and Normal Order) and one actor which is customer.

The Special Order and Normal Order use cases are extended from Order use case. So they have extends relationship. Another important point is to identify the system boundary which is shown in the picture. The actor Customer lies outside the system as it is an external user of the system.

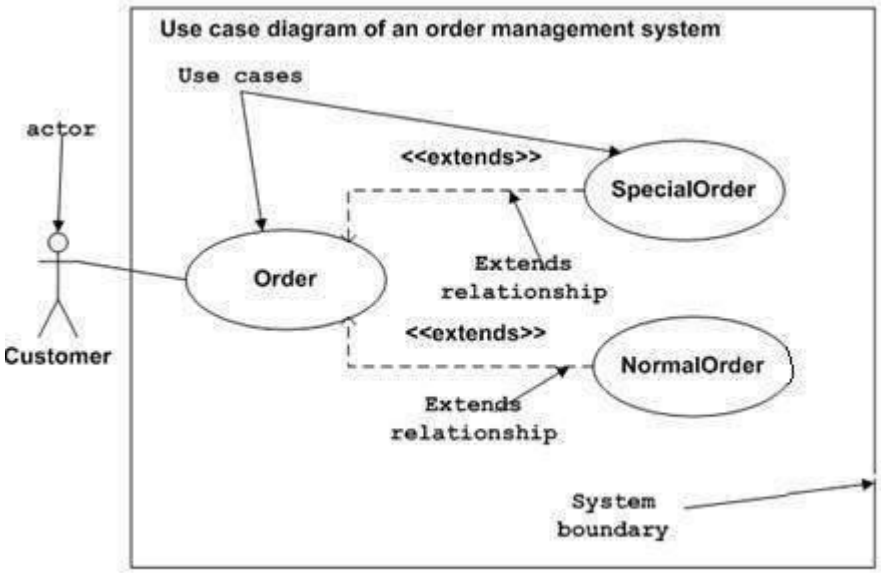


Figure: Sample Use Case diagram

Entity Relationship Diagram (ERD)

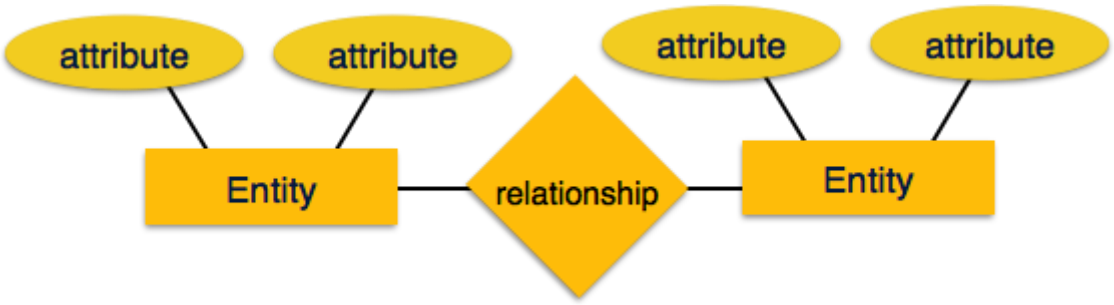
An entity-relationship diagram—otherwise known as an ERD—is a data modeling technique that creates an illustration of an information system's entities and the relationships between those entities.

3 ingredients of entity-relationship diagram

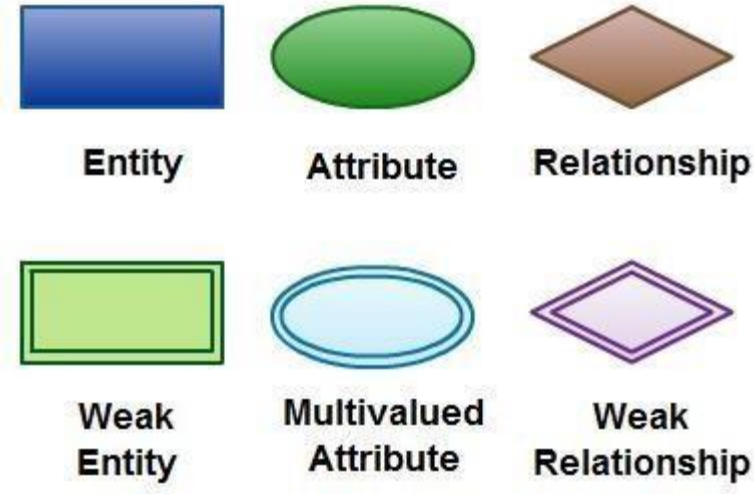
Entities, which represent people, places, items, events, or concepts.

Attributes, which represent properties or descriptive qualities of an entity. These are also known as data elements.

Relationships, which represent the link between different entities.



ER Diagram Symbols and Notations



Cardinality and Modality

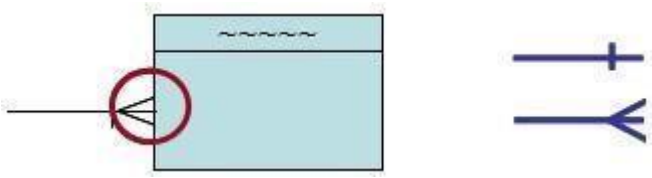
Cardinality and Modality work together to define the relationship.

- Cardinality indicates the maximum number of times an instance in one entity can be associated with instances in the related entity.
- Modality indicates the minimum number of times an instance in one entity can be associated with an instance in the related entity.

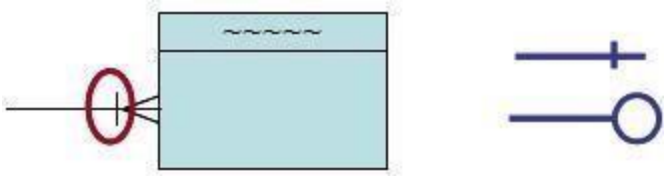
Thus, Modality is also called participation because it denotes whether or not an instance of an entity MUST participate in the relationship.

Cardinality and Modality are both shown on the relationship line by symbols. We will go over each of the symbols and how to interpret them.

Cardinality

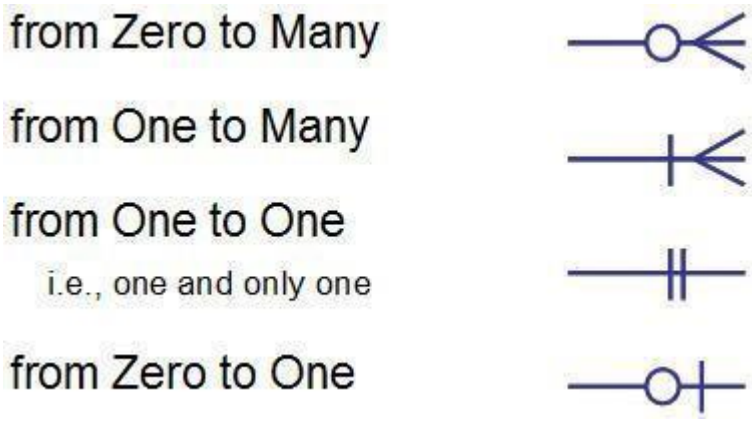


Cardinality indicates the maximum number of times an instance of one entity can be associated with instances in the related entity. Cardinality can have the values of one or many, no more detail than that. It is either one or more than one. On the relationship line, the cardinality is the closest to the entity box. The cardinality symbol in the diagram on the slide is in the red circle. Cardinality is indicated at BOTH ends of the relationship line, so there is a left to right cardinality and a right to left cardinality. **Modality**



Modality indicates the minimum number of times an instance in one entity can be associated with an instance in the related entity. Modality can have the values of zero or one, two or three are not allowed. The modality symbol is located next to the cardinality symbol, on the inside, i.e., NOT next to the entity box. A modality of one is denoted by a straight vertical line and a modality of zero is denoted by a circle. Like cardinality, modality is indicated at both ends of the relationship.

Reading Modality and Cardinality

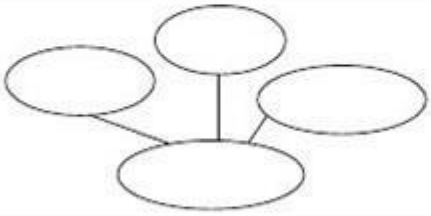


Types of Attributes · Simple attribute − Simple attributes are atomic values, which cannot be divided further. For example, a student's phone number is an atomic value of 10 digits.

- Composite attribute − Composite attributes are made of more than one simple attribute. For example, a student's complete name may have first_name and last_name.

- Derived attribute − Derived attributes are the attributes that do not exist in the physical database, but their values are derived from other attributes present in the database. For example, average_salary in a department should not be saved directly in the database, instead it can be derived. For another example, age can be derived from data_of_birth.
- Single-value attribute − Single-value attributes contain single value. For example − Social_Security_Number.

- Multi-value attribute − Multi-value attributes may contain more than one values. For example, a person can have more than one phone number, email_address, etc.

	Simple Attribute
	Multivalue Attribute
	Composite Attribute
	Derived Attribute

Types of Entity –

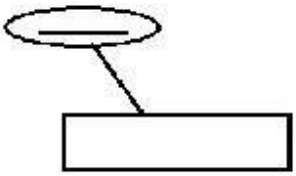
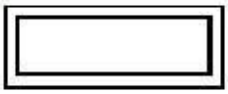
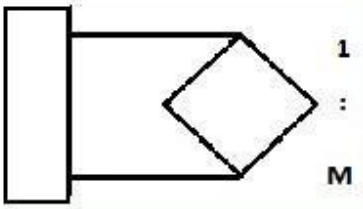
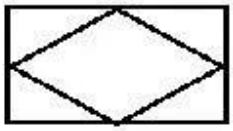
Strong Entity Types

Recursive Entity Types

Weak Entity Types

Composite Entity Types or Associative Entity Types

Notations Of different Entity Type in ER Diagram

	Entity
	Strong Entity Type
	Weak Entity Type
	Recursive Entity Type
	Composite Entity Type (or) Associative Entity

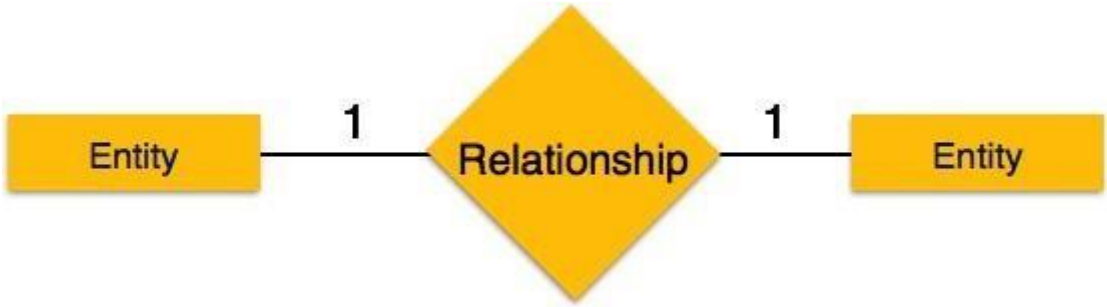
Relationship

Relationships are represented by diamond-shaped box. Name of the relationship is written inside the diamond-box. All the entities (rectangles) participating in a relationship, are connected to it by a line.

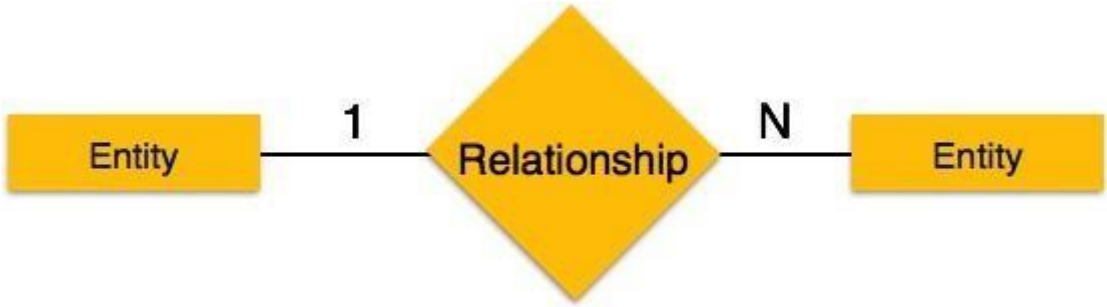
Binary Relationship and Cardinality

A relationship where two entities are participating is called a binary relationship. Cardinality is the number of instance of an entity from a relation that can be associated with the relation.

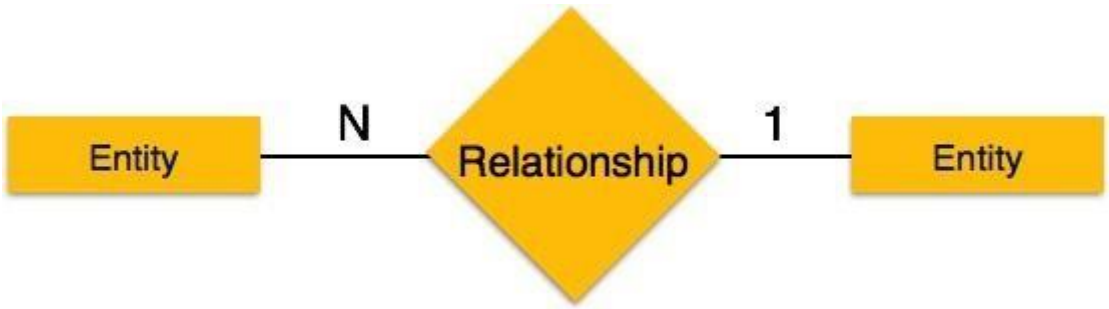
- One-to-one − When only one instance of an entity is associated with the relationship, it is marked as '1:1'. The following image reflects that only one instance of each entity should be associated with the relationship. It depicts onetoone relationship.



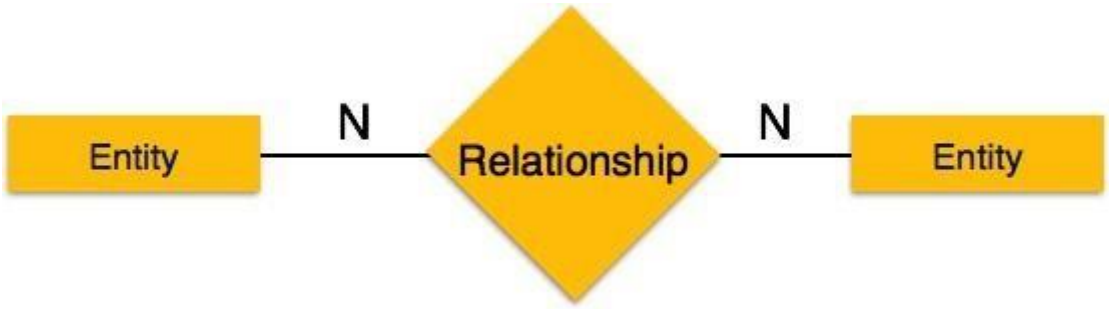
- One-to-many − When more than one instance of an entity is associated with a relationship, it is marked as '1:N'. The following image reflects that only one instance of entity on the left and more than one instance of an entity on the right can be associated with the relationship. It depicts one-tomany relationship.



- Many-to-one − When more than one instance of entity is associated with the relationship, it is marked as 'N:1'. The following image reflects that more than one instance of an entity on the left and only one instance of an entity on the right can be associated with the relationship. It depicts manyto-one relationship.



- Many-to-many − The following image reflects that more than one instance of an entity on the left and more than one instance of an entity on the right can be associated with the relationship. It depicts many-to-many relationship.



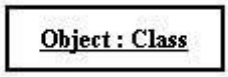
Sequence Diagrams

The Sequence Diagram shows how the objects interact with others in a particular scenario of a use case.

Basic Sequence Diagram Symbols and Notations

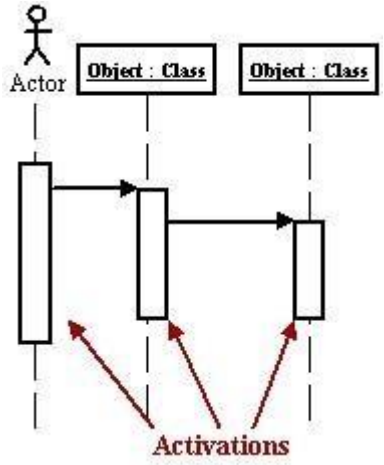
Class roles

Class roles describe the way an object will behave in context. Use the UML object symbol to illustrate class roles, but don't list object attributes.



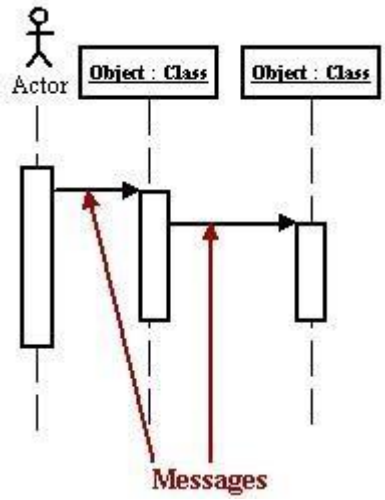
Activation

Activation boxes represent the time an object needs to complete a task.



Messages

Messages are arrows that represent communication between objects. Use half-arrowed lines to represent asynchronous messages. Asynchronous messages are sent from an object that will not wait for a response from the receiver before continuing its tasks.



Various message types for Sequence and Collaboration diagrams

Types of Messages in Sequence Diagrams

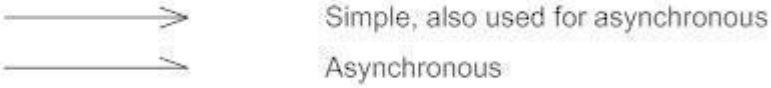
Synchronous Message

A synchronous message requires a response before the interaction can continue. It's usually drawn using a line with a solid arrowhead pointing from one object to another.



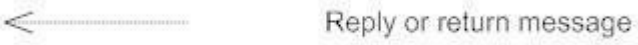
Asynchronous Message

Asynchronous messages don't need a reply for interaction to continue. Like synchronous messages, they are drawn with an arrow connecting two lifelines; however, the arrowhead is usually open and there's no return message depicted.



Reply or Return Message

A reply message is drawn with a dotted line and an open arrowhead pointing back to the original lifeline.

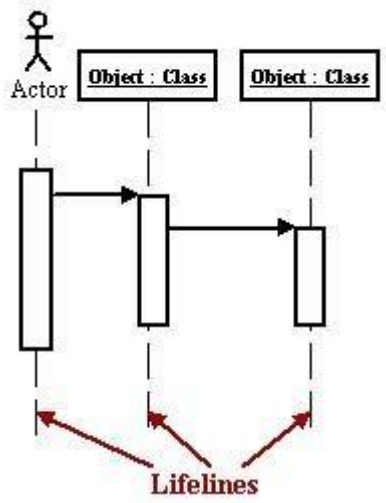


Self Message

A message an object sends to itself, usually shown as a U shaped arrow pointing back to itself.

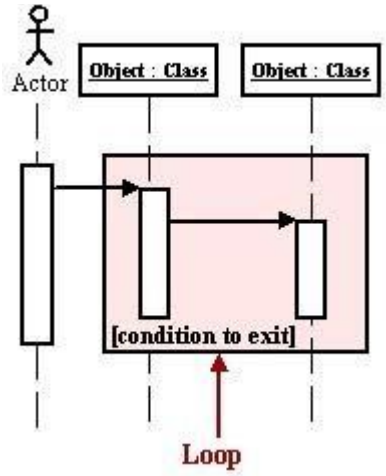
Lifelines

Lifelines are vertical dashed lines that indicate the object's presence over time.



Loops

A repetition or loop within a sequence diagram is depicted as a rectangle. Place the condition for exiting the loop at the bottom left corner in square brackets [ ].



Basic Sequence Diagram Symbols and Notations

Class Roles or Participants

Class roles describe the way an object will behave in context. Use the UML object symbol to illustrate class roles, but don't list object attributes.



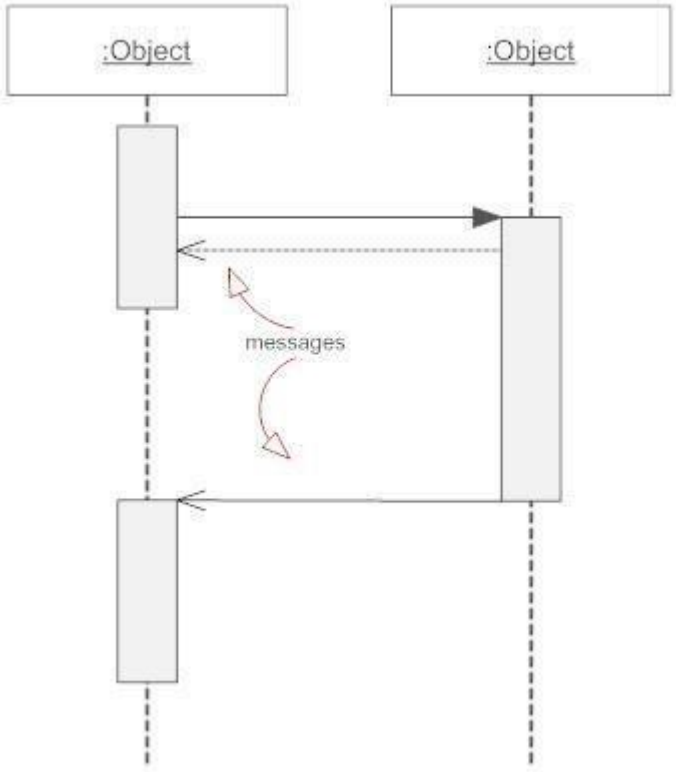
Activation or Execution Occurrence

Activation boxes represent the time an object needs to complete a task. When an object is busy executing a process or waiting for a reply message, use a thin gray rectangle placed vertically on its lifeline.



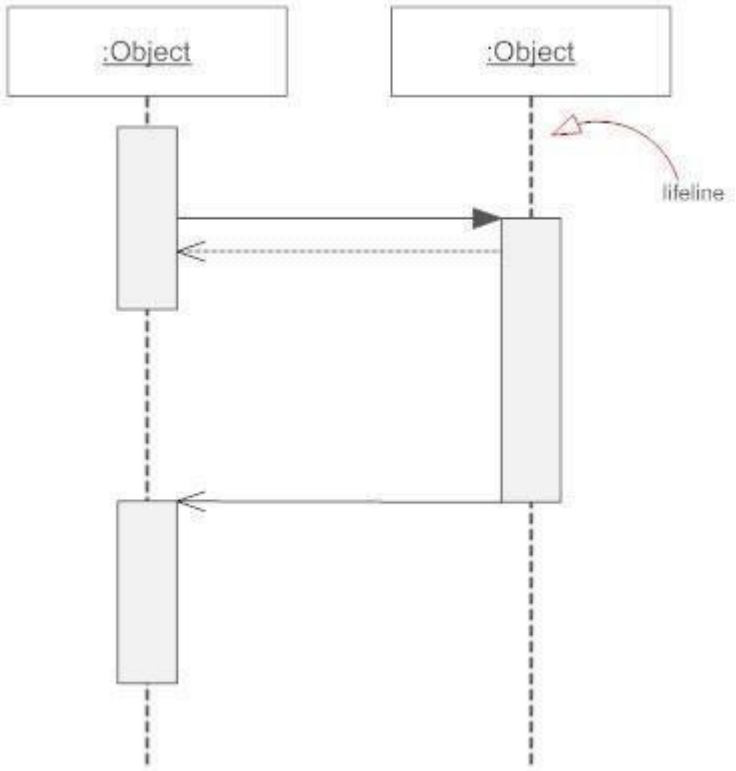
Messages

Messages are arrows that represent communication between objects. Use half-arrowed lines to represent asynchronous messages. Asynchronous messages are sent from an object that will not wait for a response from the receiver before continuing its tasks. For message types, see below.



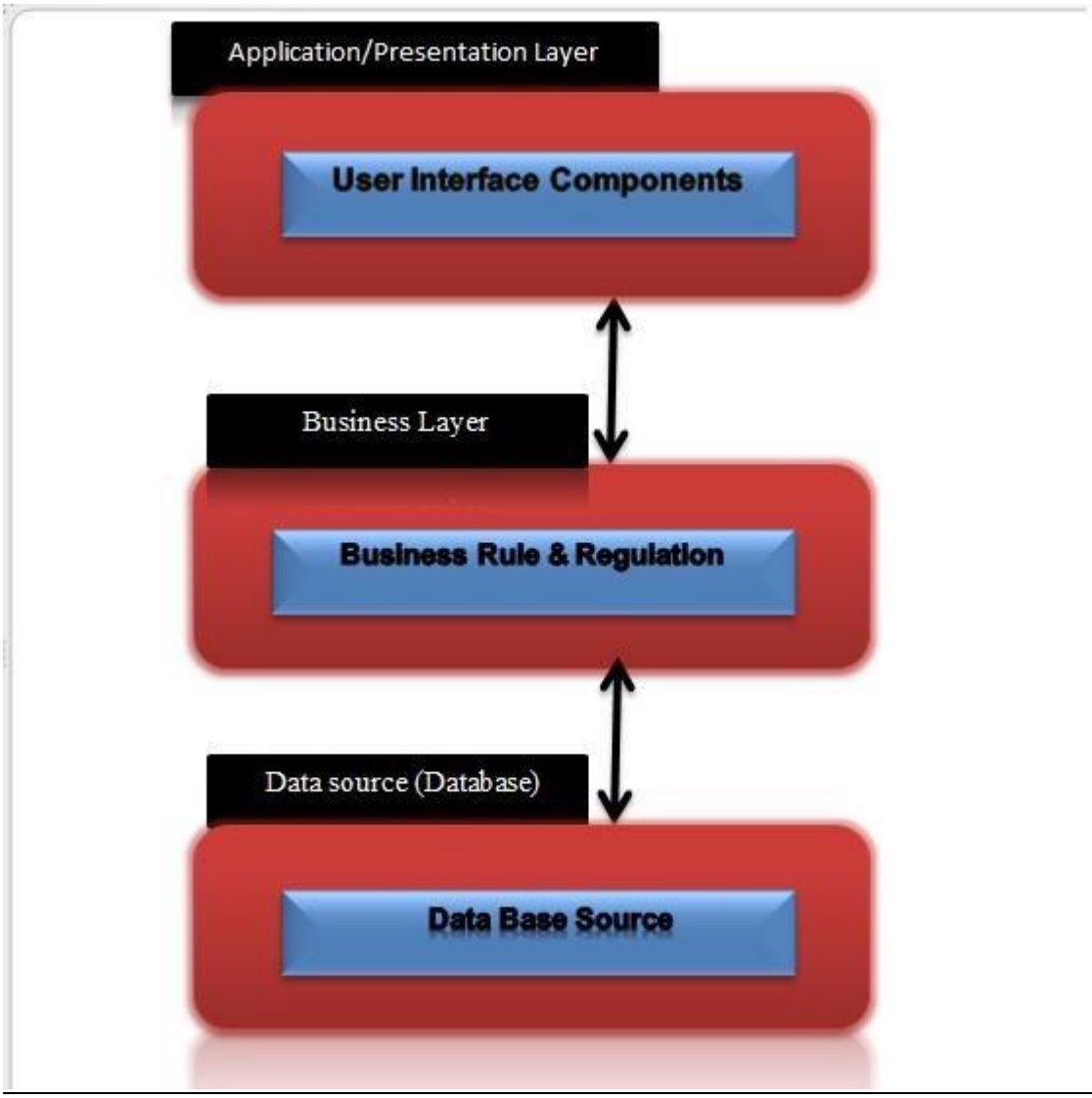
Lifelines

Lifelines are vertical dashed lines that indicate the object's presence over time.



Architecture Design Diagram

The architecture adopted is Three-tier web architecture. The architecture consists of three layers which are:



1. Presentation tier

Front End is designed in programming language like PHP, C# or Java etc...

2. Application tier

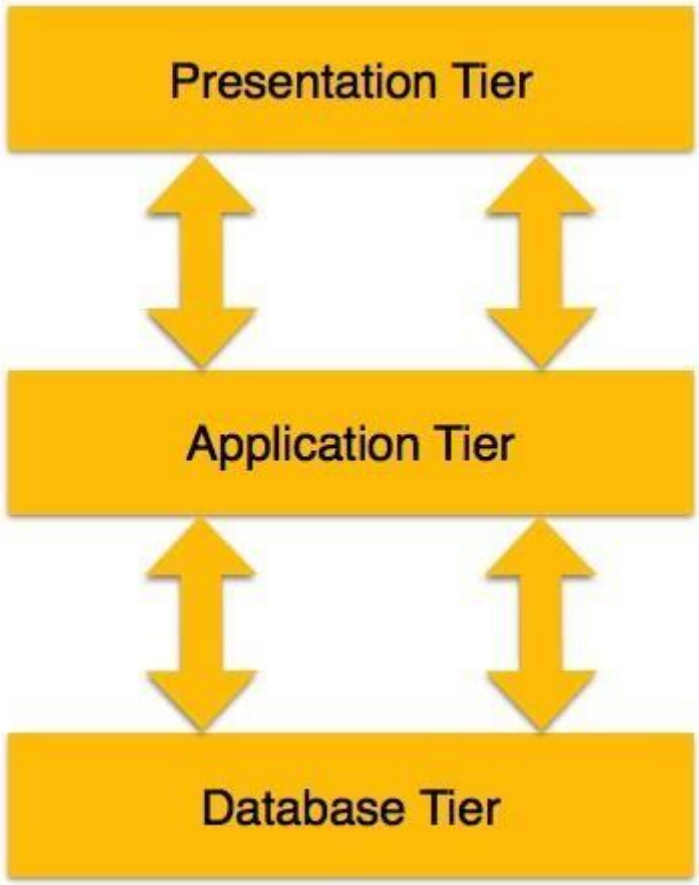
The middle dynamic content processing and generation level server layer is created in PHP, C# or Java etc...

3. Data tier

SQL or MySQL is used as back-end database server.

3-tier Architecture

A 3-tier architecture separates its tiers from each other based on the complexity of the users and how they use the data present in the database. It is the most widely used architecture to design a DBMS.



- **Database (Data) Tier** − At this tier, the database resides along with its query processing languages. We also have the relations that define the data and their constraints at this level.

- **Application (Middle) Tier** − At this tier reside the application server and the programs that access the database.

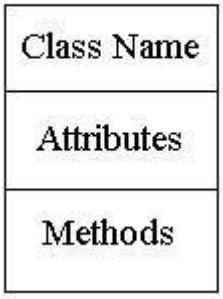
For a user, this application tier presents an abstracted view of the database. End-users are unaware of any existence of the database beyond the application. At the other end, the database tier is not aware of any other user beyond the application tier. Hence, the application layer sits in the middle and acts as a mediator between the end-user and the database.

- **User (Presentation) Tier** − End-users operate on this tier and they know nothing about any existence of the database beyond this layer. At this layer, multiple views of the database can be provided by the application. All views are generated by applications that reside in the application tier.

Class Diagram

Class diagrams are used to represents real life objects in our design by capturing its data methods and relationships of these methods with each other. These classes are basic building blocks of our object oriented system. Class diagrams are represented with white boxes which consist of three parts. Each of these parts has unique characteristics.

Syntax

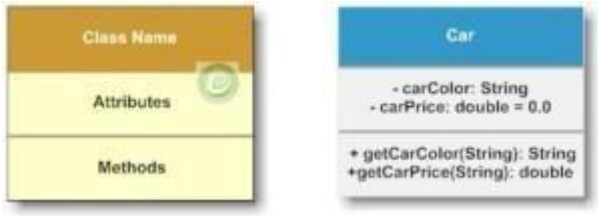


The details of these parts are as follow.

1. The upper part contains the name of the class

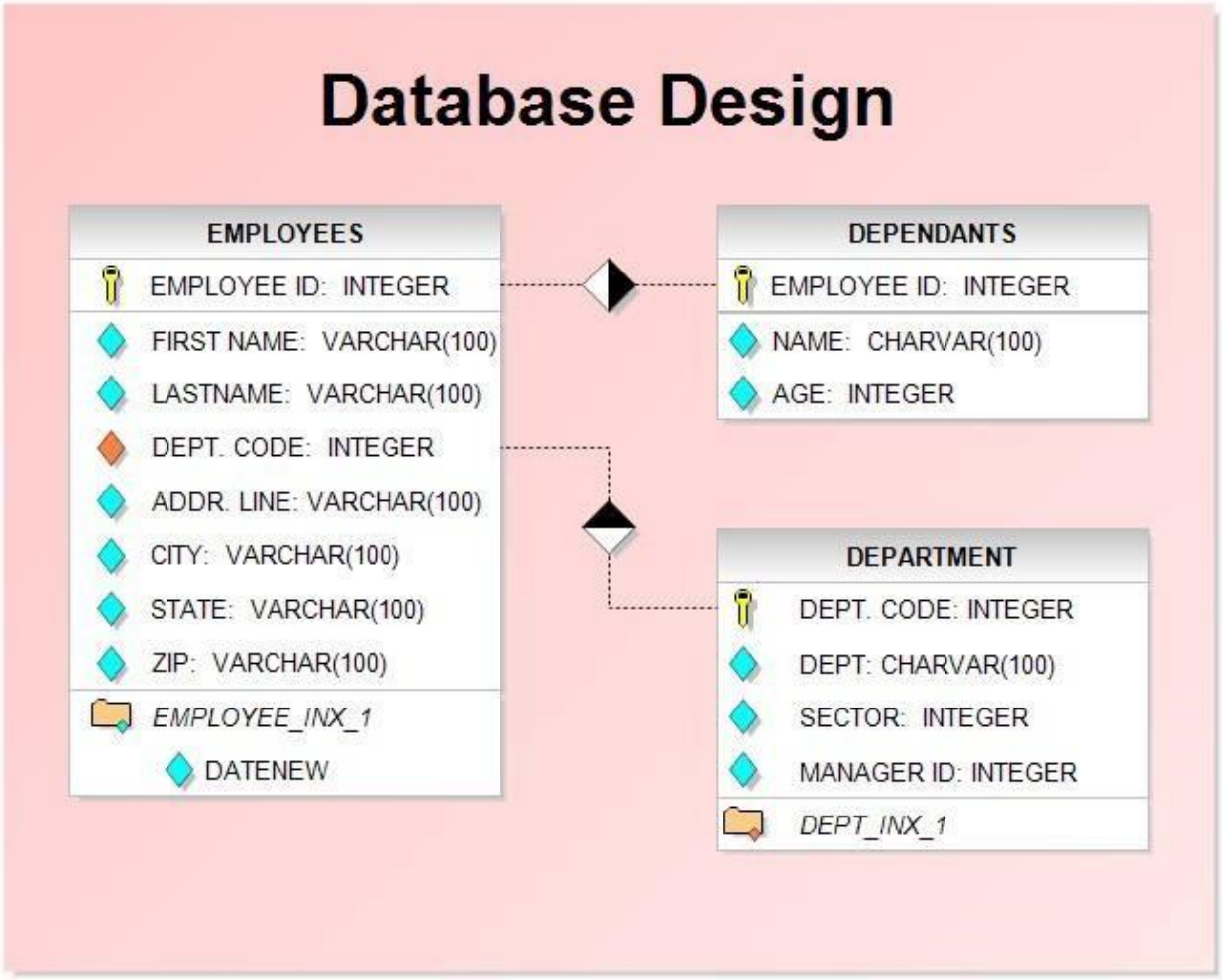
2. The next part contains the attributes of the class.
3. The third part contains the methods and operations on the class.

Class Diagram Example



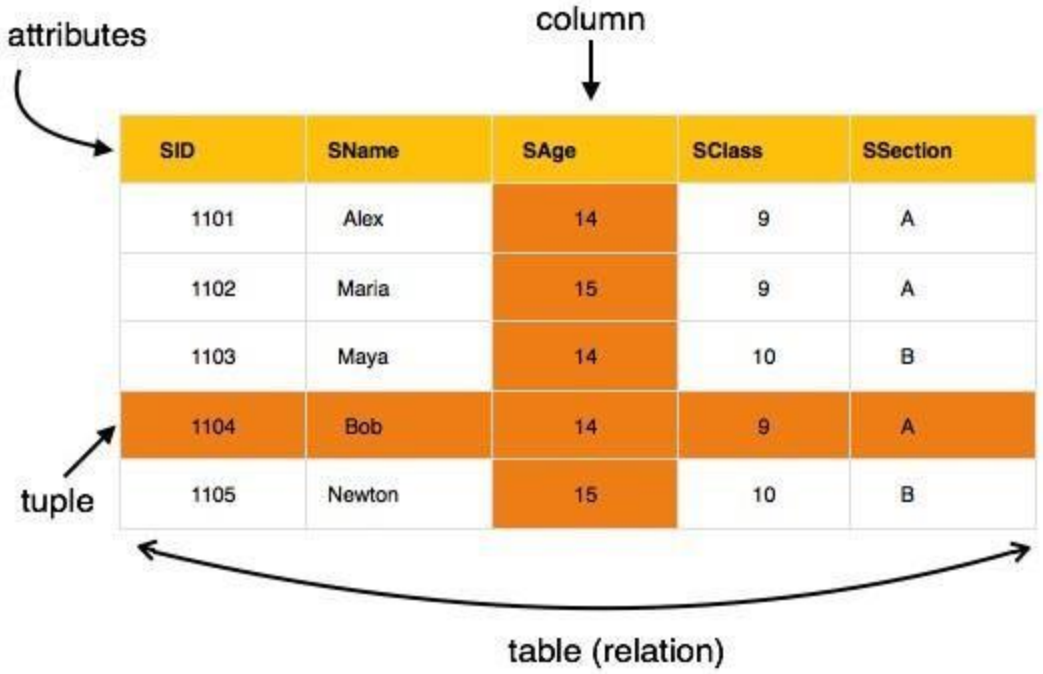
Database Design

In our daily life every object has some data associated with it. This data can be managed through data base design. Data large data can be represented by this data base design. Data base design can be represented in the form of physical, conceptual and logical.



Relational Model

The most popular data model in DBMS is the Relational Model. It is more scientific a model than others. This model is based on first-order predicate logic and defines a table as an n-ary relation.



The main highlights of this model are −

- Data is stored in tables called relations.
- Relations can be normalized.
- In normalized relations, values saved are atomic values.
- Each row in a relation contains a unique value.
- Each column in a relation contains values from a same domain.

Relation Data Model Concepts Tables − In relational data model, relations are saved in the format of Tables. This format stores the relation among entities. A table has rows and columns, where rows represents records and columns represent the attributes.

Tuple − A single row of a table, which contains a single record for that relation is called a tuple.

Relation instance − A finite set of tuples in the relational database system represents relation instance. Relation instances do not have duplicate tuples.

Relation schema − A relation schema describes the relation name (table name), attributes, and their names.

Relation key − Each row has one or more attributes, known as relation key, which can identify the row in the relation (table) uniquely.

Attribute domain − Every attribute has some pre-defined value scope, known as attribute domain.

Database Management System or DBMS

Database Management System or DBMS in short refers to the technology of storing and retrieving users' data with utmost efficiency along with appropriate security measures. This tutorial explains the basics of DBMS such as its architecture, data models, data schemas, data independence, E-R model, relation model, relational database design, and storage and file structure and much more.

Database Management System: is the set of programs or system which is used to create and maintain database.

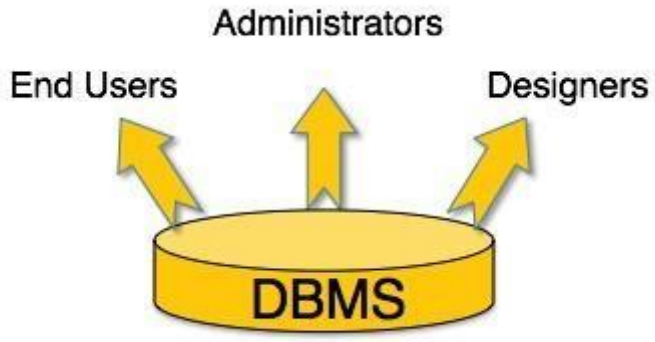
Data: is the collection of raw, facts and figures like college admission form consists of data.

Database: is organized collection of related data.

A database management system stores data in such a way that it becomes easier to retrieve, manipulate, and produce information.

Users

A typical DBMS has users with different rights and permissions who use it for different purposes. Some users retrieve data and some back it up. The users of a DBMS can be broadly categorized as follows −



- **Administrators** − Administrators maintain the DBMS and are responsible for administrating the database. They are responsible to look after its usage and by whom it should be used. They create access profiles for users and apply limitations to maintain isolation and force security. Administrators also look after DBMS resources like system license, required tools, and other software and hardware related maintenance.

- **Designers** − Designers are the group of people who actually work on the designing part of the database. They keep a close watch on what data should be kept and in what format. They identify and design the whole set of entities, relations, constraints, and views.

- **End Users** − End users are those who actually reap the benefits of having a DBMS. End users can range from

simple viewers who pay attention to the logs or market rates to sophisticated users such as business analysts.

Keys

Super Key

Super key is a set of one or more than one keys that can be used to identify a record uniquely in a table. Example: Primary key, Unique key, Alternate key are subset of Super Keys.

Candidate Key

A Candidate Key is a set of one or more fields/columns that can identify a record uniquely in a table. There can be multiple Candidate Keys in one table. Each Candidate Key can work as Primary Key.

Example: In below diagram ID, RollNo and EnrollNo are Candidate Keys since all these three fields can be work as Primary Key.

Primary Key

Primary key is a set of one or more fields/columns of a table that uniquely identify a record in database table. It can not accept null, duplicate values. Only one Candidate Key can be Primary Key.

Alternate key

A Alternate key is a key that can be work as a primary key. Basically it is a candidate key that currently is not primary key.

Example: In below diagram RollNo and EnrollNo becomes Alternate Keys when we define ID as Primary Key.

Composite/Compound Key

Composite Key is a combination of more than one fields/columns of a table. It can be a Candidate key, Primary key.

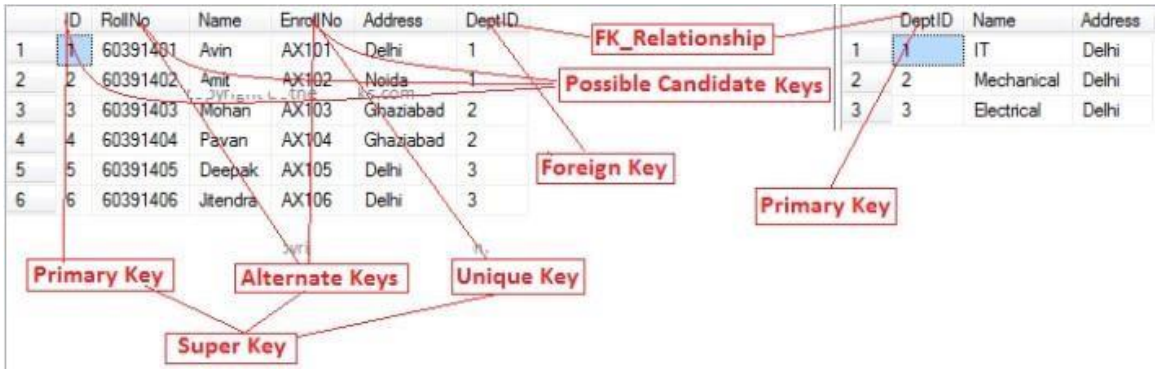
Unique Key

Uniquekey is a set of one or more fields/columns of a table that uniquely identify a record in database table. It is like Primary key but it can accept only one null value and it can not have duplicate values. For more help refer the article Difference between primary key and unique key.

Foreign Key

Foreign Key is a field in database table that is Primary key in another table. It can accept multiple null, duplicate values. For more help refer the article Difference between primary key and foreign key.

Example : We can have a DeptID column in the Employee table which is pointing to DeptID column in a department table where it a primary key.



Note: Practically in database, we have only three types of keys Primary Key, Unique Key and Foreign Key. Other types of keys are only concepts of RDBMS that we need to know.

SQL Overview

SQL is a programming language for Relational Databases. It is designed over relational algebra and tuple relational calculus. SQL comes as a package with all major distributions of RDBMS.

SQL comprises both data definition and data manipulation languages. Using the data definition properties of SQL, one can design and modify database schema, whereas data manipulation properties allows SQL to store and retrieve data from database.

SQL Commands:

The standard SQL commands to interact with relational databases are CREATE, SELECT, INSERT, UPDATE, DELETE and DROP. These commands can be classified into groups based on their nature:

DDL - Data Definition Language:

Command	Description
CREATE	Creates a new table, a view of a table, or other object in database
ALTER	Modifies an existing database object, such as a table.

DROP	Deletes an entire table, a view of a table or other object in the database.

DML - Data Manipulation Language:

Command	Description
SELECT	Retrieves certain records from one or more tables
INSERT	Creates a record
UPDATE	Modifies records
DELETE	Deletes records

DCL - Data Control Language:

Command	Description
GRANT	Gives a privilege to user
REVOKE	Takes back privileges granted from user

Data Definition Language

The DDL section is used for creating database objects, such as tables. In practice, people often use a GUI for creating tables and

so on, so it is less common to hand-write DDL statements than it used to be.

SQL uses the following set of commands to define database schema −

CREATE

Creates new databases, tables and views from RDBMS.

For example −

```
Create database tutorialspoint;
Create table article;
Create view for_students;
```

DROP

Drops commands, views, tables, and databases from RDBMS.

For example−

```
Drop object_type object_name;
Drop database tutorialspoint;
Drop table article;
Drop view for_students;
```

ALTER

Modifies database schema.

```
Alter object_type object_name parameters;
```

For example−

```
Alter table article add subject varchar;
```

This command adds an attribute in the relation article with the name subject of string type.

Difference between Drop, Truncate and Delete

DELETE

DELETE removes some rows if WHERE clause is used

DROP

Removes a table from the database. Table structures, indexes, privileges, constraints will also be removed.

TRUNCATE

It Removes all rows from a table, but the table structures and its columns, constraints, indexes remains.

Data Manipulation Language

The DML section is used to manipulate the data such as querying it. While is also common to use a query builder to create queries, people do still hand-craft DML statements, such as queries.

SQL is equipped with data manipulation language (DML). DML modifies the database instance by inserting, updating and deleting its data. DML is responsible for all froms data modification in a database. SQL contains the following set of commands in its DML section −

- SELECT/FROM/WHERE
- INSERT INTO/VALUES
- UPDATE/SET/WHERE
- DELETE FROM/WHERE

These basic constructs allow database programmers and users to enter data and information into the database and retrieve efficiently using a number of filter options.

SELECT/FROM/WHERE

- SELECT − This is one of the fundamental query command of SQL. It is similar to the projection operation of relational algebra. It selects the attributes based on the condition described by WHERE clause.

- FROM − This clause takes a relation name as an argument from which attributes are to be selected/projected. In case more than one relation names are given, this clause corresponds to Cartesian product.

- WHERE − This clause defines predicate or conditions, which must match in order to qualify the attributes to be projected.

For example −

```
Select author_name
From book_author
Where age > 50;
```

This command will yield the names of authors from the relation book_authorwhose age is greater than 50.

INSERT INTO/VALUES

This command is used for inserting values into the rows of a table (relation).

Syntax−

```
INSERT INTO table (column1 [, column2, column3 ... ]) VALUES
(value1 [, value2, value3 ... ])
```

Or

```
INSERT INTO table VALUES (value1, [value2, ... ])
```

For example −

```
INSERT INTO tutorialspoint (Author, Subject) VALUES
("anonymous", "computers");
```

UPDATE/SET/WHERE

This command is used for updating or modifying the values of columns in a table (relation).

Syntax −

```
UPDATE table_name SET column_name = value [, column_name =
value ...] [WHERE condition]
```

For example −

```
UPDATE tutorialspoint SET Author="webmaster" WHERE
Author="anonymous";
```

DELETE/FROM/WHERE

This command is used for removing one or more rows from a table (relation).

Syntax −

```
DELETE FROM table_name [WHERE condition];
```

For example −

```
DELETE FROM
tutorialspoints    WHERE Author="unknown";
```

CRUD Operations

C stand for create
R stand for read
U stand for update
D stand for delete

Normalization

If a database design is not perfect, it may contain anomalies, which are like a bad dream for any database administrator. Managing a database with anomalies is next to impossible.

- Update anomalies − If data items are scattered and are not linked to each other properly, then it could lead to strange situations. For example, when we try to update one data item having its copies scattered over several places, a few instances get updated properly while a few others are left with old values. Such instances leave the database in an inconsistent state.

- Deletion anomalies − We tried to delete a record, but parts of it was left undeleted because of unawareness, the data is also saved somewhere else.

- Insert anomalies − We tried to insert data in a record that does not exist at all.

Normalization is a method to remove all these anomalies and bring the database to a consistent state.

First Normal Form

First Normal Form is defined in the definition of relations (tables) itself. This rule defines that all the attributes in a relation must have atomic domains. The values in an atomic domain are indivisible units.

Course	Content
Programming	Java, c++
Web	HTML, PHP, ASP

We re-arrange the relation (table) as below, to convert it to First Normal Form.

Course	Content
Programming	Java
Programming	c++
Web	HTML
Web	PHP
Web	ASP

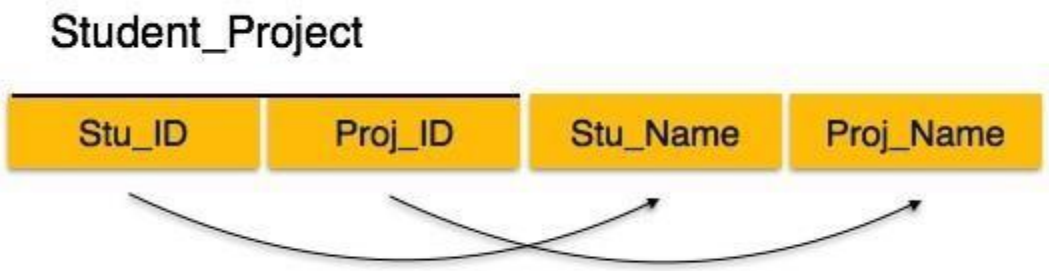
Each attribute must contain only a single value from its predefined domain.

Second Normal Form

Before we learn about the second normal form, we need to understand the following −

- Prime attribute − An attribute, which is a part of the primekey, is known as a prime attribute.
- Non-prime attribute − An attribute, which is not a part of the prime-key, is said to be a non-prime attribute.

If we follow second normal form, then every non-prime attribute should be fully functionally dependent on prime key attribute. That is, if X → A holds, then there should not be any proper subset Y of X, for which Y → A also holds true.



We see here in Student_Project relation that the prime key attributes are Stu_ID and Proj_ID. According to the rule, non-key attributes, i.e. Stu_Name and Proj_Name must be dependent upon both and not on any of the prime key attribute individually. But we find that Stu_Name can be identified by Stu_ID and Proj_Name can be identified by Proj_ID independently. This is calledpartial dependency, which is not allowed in Second Normal Form.



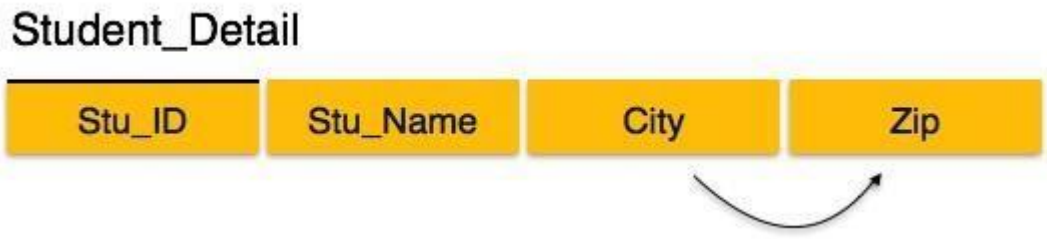


We broke the relation in two as depicted in the above picture. So there exists no partial dependency.

Third Normal Form

For a relation to be in Third Normal Form, it must be in Second Normal form and the following must satisfy −

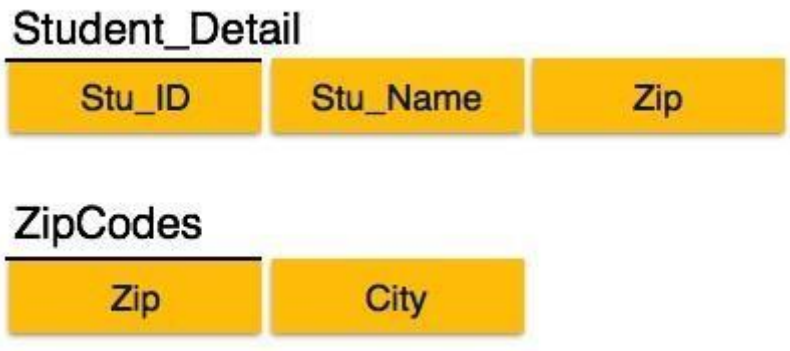
- No non-prime attribute is transitively dependent on prime key attribute.
- For any non-trivial functional dependency, X → A, then either − ○ X is a superkey or, ○ A is prime attribute.



We find that in the above Student_detail relation, Stu_ID is the key and only prime key attribute. We find that City can be identified by Stu_ID as well as Zip itself. Neither Zip is a superkey nor is City a prime attribute. Additionally, Stu_ID → Zip → City, so there exists transitive dependency.

To bring this relation into third normal form, we break the relation

into two relations as follows −



Anomalies

Anomalies are inconvenient or error-prone situation arising when we process the tables. There are three types of anomalies:

Update Anomalies

Delete Anomalies

Insert Anomalies

Insert Anomalies

An Insert Anomaly occurs when certain attributes cannot be inserted into the database without the presence of other attributes. For example this is the converse of delete anomaly - we can't add a new course unless we have at least one student enrolled on the course.

StudentNum	CourseNum	Student Name	Address	Course
S21	9201	Jones	Edinburgh	Accounts
S21	9267	Jones	Edinburgh	Accounts
S24	9267	Smith	Glasgow	physics
S30	9201	Richards	Manchester	Computing
S30	9322	Richards	Manchester	Maths

Delete Anomalies

A Delete Anomaly exists when certain attributes are lost because of the deletion of other attributes. For example, consider what happens if Student S30 is the last student to leave the course - All information about the course is lost.

StudentNum	CourseNum	Student Name	Address	Course
S21	9201	Jones	Edinburgh	Accounts
S21	9267	Jones	Edinburgh	Accounts
S24	9267	Smith	Glasgow	physics
S30	9201	Richards	Manchester	Computing
S30	9322	Richards	Manchester	Maths

Update Anomalies

An Update Anomaly exists when one or more instances of duplicated data is updated, but not all. For example, consider Jones moving address - you need to update all instances of Jones's address.

StudentNum	CourseNum	Student Name	Address	Course
S21	9201	Jones	Edinburgh	Accounts
S21	9267	Jones	Edinburgh	Accounts
S24	9267	Smith	Glasgow	physics
S30	9201	Richards	Manchester	Computing
S30	9322	Richards	Manchester	Maths

Data Integrity

Integrity ensures that the data in a database is both accurate and complete, in other words, that the data makes sense. There are at least five different types of integrity that need to be considered:

Domain constraints

Entity integrity

Column constraints

User-defined integrity constraints

Referential integrity

The data analysis stage will identify the requirements of these.

Domain Constraints

A domain is defined as the set of all unique values permitted for an attribute. For example, a domain of Date is the set of all possible valid dates, a domain of Integer is all possible whole numbers, and a domain of day-of-week is Monday, Tuesday ... Sunday.

Entity Integrity

Entity integrity is concerned with ensuring that each row of a table has a unique and non-null primary key value; this is the same as saying that each row in a table represents a single instance of the entity type modelled by the table. A requirement of E F Codd in his seminal paper is that a primary key of an entity, or any part of it, can never take a null value.

Column Constraints

During the data analysis phase, business rules will identify any column constraints. For example, a salary cannot be negative; an employee number must be in the range 1000 - 2000, etc.

UserDefined Integrity Constraints

Business rules may dictate that when a specific action occurs, further actions should be triggered. For example, deletion of a record automatically writes that record to an audit table.

Referential Integrity

Referential integrity is with the relationships between the tables of a database, ie that the data of one table does not contradict the data of another table. Specifically, every foreign key value in a table must have a matching primary key value in the related table. This is the most common type of integrity constraint. This is used to manage the relationships between primary and foreign keys.

Joins

Join is a combination of a Cartesian product followed by a selection process. A Join operation pairs two tuples from different relations, if and only if a given join condition is satisfied.

We will briefly describe various join types in the following sections.

Theta (θ) Join

Theta join combines tuples from different relations provided they satisfy the theta condition. The join condition is denoted by the symbol θ.

Notation

$$R1 \ _{\theta} \ R2$$

R1 and R2 are relations having attributes (A1, A2, .., An) and (B1, B2,.. ,Bn) such that the attributes don't have anything in common, that is $R1 \cap R2 = \Phi$.

Equijoin

When Theta join uses only equality comparison operator, it is said to be equijoin. The above example corresponds to equijoin.

Natural Join ( )

Natural join does not use any comparison operator. It does not concatenate the way a Cartesian product does. We can perform a Natural Join only if there is at least one common attribute that exists between two relations. In addition, the attributes must have the same name and domain.

Natural join acts on those matching attributes where the values of attributes in both the relations are same. **Outer Joins**

Theta Join, Equijoin, and Natural Join are called inner joins. An inner join includes only those tuples with matching attributes and the rest are discarded in the resulting relation. Therefore, we need to use outer joins to include all the tuples from the participating relations in the resulting relation. There are three kinds of outer joins − left outer join, right outer join, and full outer join.

Left Outer Join(R ⟕ S)

All the tuples from the Left relation, R, are included in the resulting relation. If there are tuples in R without any matching tuple in the Right relation S, then the S-attributes of the resulting relation are made NULL.

Right Outer Join: (R ⟖ S)

All the tuples from the Right relation, S, are included in the resulting relation. If there are tuples in S without any matching tuple in R, then the R-attributes of resulting relation are made NULL.

Full Outer Join: (R ⟗ S)

All the tuples from both participating relations are included in the resulting relation. If there are no matching tuples for both relations, their respective unmatched attributes are made NULL.

Indexing

Indexing is a data structure technique to efficiently retrieve records from the database files based on some attributes on which the indexing has been done. Indexing in database systems is similar to what we see in books.

Indexing is defined based on its indexing attributes. Indexing can be of the following types −

- Primary Index − Primary index is defined on an ordered data file. The data file is ordered on a key field. The key field is generally the primary key of the relation.
- Secondary Index − Secondary index may be generated from a field which is a candidate key and has a unique value in every record, or a non-key with duplicate values.
- Clustering Index − Clustering index is defined on an ordered data file. The data file is ordered on a non-key field.